

Lab#7 – White-box testing

วัตถุประสงค์การเรียนรู้

1. ผู้เรียนสามารถออกแบบการทดสอบแบบ White-box testing ได้
2. ผู้เรียนสามารถวิเคราะห์ปัญหาด้วย Control flow graph ได้
3. ผู้เรียนสามารถออกแบบกรณีทดสอบโดยคำนึงถึง Line coverage ได้
4. ผู้เรียนสามารถออกแบบกรณีทดสอบโดยคำนึงถึง Block coverage ได้
5. ผู้เรียนสามารถออกแบบกรณีทดสอบโดยคำนึงถึง Branch coverage ได้
6. ผู้เรียนสามารถออกแบบกรณีทดสอบโดยคำนึงถึง Condition coverage ได้
7. ผู้เรียนสามารถออกแบบกรณีทดสอบโดยคำนึงถึง Branch and Condition coverage ได้

โจทย์: CLUMP COUNTS

Clump counts (<https://codingbat.com/prob/p193817>) เป็นโปรแกรมที่ใช้ในการนับการเกาะกลุ่มกันของข้อมูลภายใน Array โดยการเกาะกลุ่มกันจะนับสมาชิกใน Array ที่อยู่ติดกันและมีค่าเดียวกันตั้งแต่สองตัวขึ้นไปเป็นหนึ่งกลุ่ม เช่น

[1, 2, 2, 3, 4, 4] → 2

[1, 1, 2, 1, 1] → 2

[1, 1, 1, 1, 1] → 1

ซอร์สโค้ดที่เขียนขึ้นเพื่อนับจำนวนกลุ่มของข้อมูลที่เกาะอยู่ด้วยกันอยู่ที่

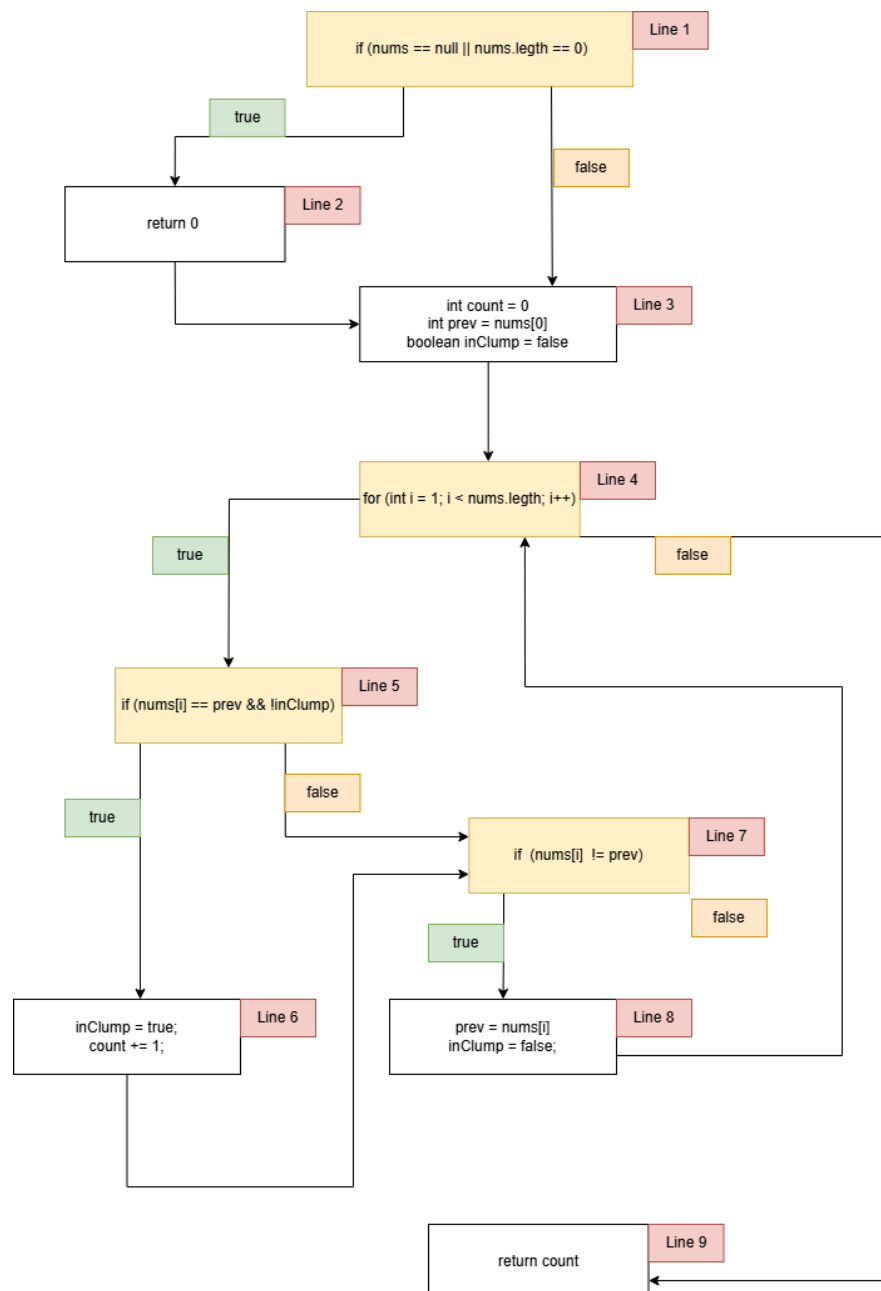
<https://github.com/ChitsuthaCSKKU/SQA/tree/2025/Assignment/Lab7>

โดยที่ nums เป็น Array ที่ใช้ในการสนับสนุนการนับกลุ่มของข้อมูล (Clump) ทำให้ nums เป็น Array ที่จะต้องมีค่าเป็น Null และมีความยาวมากกว่า 0 เสมอ หาก nums ไม่เป็นไปตามเงื่อนไขที่กำหนดนี้ โปรแกรมจะ return ค่า 0 แทนการ return จำนวนกลุ่มของข้อมูล

แบบฝึกปฏิบัติที่ 7.1 CONTROL FLOW GRAPH

จากโจทย์และ Source code ที่กำหนดให้ (CountWordClumps.java) ให้เขียน Control Flow Graph (CFG) ของเมธอด countClumps() จากนั้นให้ระบุ Branch และ Condition ทั้งหมดที่พบใน CFG ให้ครบถ้วน

ตอบ



CP353201 Software Quality Assurance (1/2568)

ผศ.ดร.ชิตสุธา สุ่มเล็ก

Lab instruction

Branch:

1. check(if (nums == null || nums.length == 0))
- 2.True branch: return 0
- 3.False branch: Continues
- 4.For loop condition (i < nums.length)
- 5.True branch: enters loop body
- 6.False branch: exits loop to return count
- 7.Inside loop: if (num[i] == prev && !inClump)
- 8.True branch: sets inClump = true, count + 1 , next condition
- 9.False branch: skips to next condition
- 10.Inside loops: if (nums[i] != prev)
- 11.True branch: updates prev,sets inClump = false
- 12.False branch: Continues loop

Condition:

nums == null

nums.length == 0

i < nums.length

nums[i] prev && !inClump

nums[i] != prev

CP353201 Software Quality Assurance (1/2568)

ผศ.ดร.ชิตสุธา สุ่มเล็ก

Lab instruction

แบบฝึกปฏิบัติที่ 7.2 LINE COVERAGE

1. จาก Control Flow Graph (CFG) ของเมธอด countClumps() ในข้อที่ 1
ให้ออกแบบกรณีทดสอบเพื่อให้ได้ Line coverage = 100%
2. เขียนกรณีทดสอบที่ได้ พร้อมระบุบรรทัดที่ถูกตรวจสอบทั้งหมด
3. แสดงวิธีการคำนวณค่า Line coverage

ตอบ

Test Case No.	Input(s)	Expected Result(s)	Path and Branch
LC01	{}	0	Line No.:1,2
LC02	{1,2,3}	0	Line No.:1,2
LC03	{1,1}	1	Line No.:1,3,4,5,6,7,9
LC04	{1,2}	0	Line No: 1,3,4,5,7,8,9

Line coverage = $(9/9) \times 100\% = 100\%$

แบบฝึกปฏิบัติที่ 7.3 BLOCK COVERAGE

1. จาก Control Flow Graph (CFG) ของเมธอด countClumps() ในข้อที่ 1
ให้ออกแบบกรณีทดสอบเพื่อให้ได้ Block coverage = 100%
2. เขียนกรณีทดสอบที่ได้ พร้อมระบุ Block ที่ถูกตรวจสอบทั้งหมด
3. แสดงวิธีการคำนวณค่า Block coverage

ตอบ

Test Case No.	Input(s)	Expected Result(s)	Path and Branch
---------------	----------	--------------------	-----------------

CP353201 Software Quality Assurance (1/2568)

ผศ.ดร.ชิตสุธา สุ่มเล็ก

Lab instruction

BC01	null	0	Block: Line 1, Line 2
BC02	{}	0	Block:Line 1, Line2
BC03	{1,1}	1	Block:Line 1 → 3 → 4 → 5(true) → 6 → 7(false) → 9
BC04	{1,2}	0	Block:Line 1 → 3 → 4 → 5(false) → 7(true) → 8 → 9

Block coverage = $(9/9) \times 100\% = 100\%$

แบบฝึกปฏิบัติที่ 7.3 BRANCH COVERAGE

4. จาก Control Flow Graph (CFG) ของเมธอด countClumps() ในข้อที่ 1
ให้ออกแบบกรณีทดสอบเพื่อให้ได้ Branch coverage = 100%

5. เขียนกรณีทดสอบที่ได้ พร้อมระบุ Path และ Branch ที่ถูกตรวจสอบทั้งหมด

6. แสดงวิธีการคำนวณค่า Branch coverage

ตอบ

Test Case No.	Input(s)	Expected Result(s)	Path and Branch
BC01	null	0	Path:Line 1 → return Branch:Line 1(true)
BC02	{}	0	Path:Line 1 → return Branch:Line 1(false)

CP353201 Software Quality Assurance (1/2568)

ผศ.ดร.ชิตสุธา สุ่มเล็ก

Lab instruction

BC03	{1,1}	1	Path:Line 1(false) → Line 5(true) → Line 6 → Line 7(false) → return Branch:Line 2(true), Line 3(false)
BC04	{1,2}	0	Path:Line 1(false) → Line 5(false) → Line 7(true) → return Branch:Line 2(false), Line3(true)

Branch coverage = $(6/6) \times 100\% = 100\%$

แบบฝึกปฏิบัติที่ 7.4 CONDITION COVERAGE

1. จาก Control Flow Graph (CFG) ของเมธอด countClumps() ในข้อที่ 1 ให้ออกแบบกรณีทดสอบเพื่อให้ได้ Condition coverage = 100%
2. เขียนกรณีทดสอบที่ได้ พร้อมระบุ Path และ Condition ที่ถูกตรวจสอบทั้งหมด เช่น Condition A = T และ Condition B = F
3. แสดงวิธีการคำนวณค่า Condition coverage

ตอบ

Test Case No.	Input(s)	Expected Result(s)	Path and Condition
CC01	null	0	nums == null(true)

CP353201 Software Quality Assurance (1/2568)

ผศ.ดร.ชิตสุธา สุ่มเล็ก

Lab instruction

CC02	{}	0	nums == null(false), nums.length == 0(true)
CC03	{1}	0	nums.length == 0(false), no loop
CC04	{1,1}	1	nums[i] == prev(true), !inClump(true)
CC05	{1,1,2}	1	nums[i] == prev(true), !inClump(false), nums[i] != prev(true)
CC06	{1,2,1}	0	nums[i] == prev(false), nums[i] != prev(true)

Condition coverage = $(10/10) \times 100\% = 100\%$

แบบฝึกปฏิบัติที่ 7.5 BRANCH AND CONDITION COVERAGE (C/DC COVERAGE)

1. จาก Control Flow Graph (CFG) ของเมธอด countClumps() ในข้อที่ 1 ให้ออกแบบกรณีทดสอบให้ได้ C/DC coverage = 100%
2. เขียนกรณีทดสอบที่ได้ พร้อมระบุ Path, Branch, และ Condition ที่ถูกตรวจสอบทั้งหมด

CP353201 Software Quality Assurance (1/2568)

ผศ.ดร.ชิตสุธา สุ่มเล็ก

Lab instruction

3. แสดงวิธีการคำนวณค่า C/DC coverage

4. เขียนโค้ดสำหรับทดสอบตามกรณีทดสอบที่ออกแบบไว้ด้วย JUnit และบันทึกผลการทดสอบ

ตอบ

Test Case No.	Input(s)	Expected Result(s)	Actual Result(s)	Path, Branch, and Condition
TC01	null	0	0 Pass	Path: Line1 → Line2 Decision1: true Condition1.1: true
TC02	{}	0	0 Pass	Path: Line1 → Line2 Decision1: true Condition1.1: false
TC03	{1}	0	0 Pass	Path: Line1 → Line3 → Line9 Decision1: false
TC04	{1,1}	1	1 Pass	Path: Line1 → Line3 → Line4 → Line5 → Line6 → Line9

CP353201 Software Quality Assurance (1/2568)

ผศ.ดร.ชิตสุธา สุ่มเล็ก

Lab instruction

				Decision1: false Decision2: true Condition2.1: true
TC05	{1,1,2}	1	1 Pass	Path: Line1 → Line3 → Line4 → Line5 → Line6 → Line7 → Line8 → Line9 Decision1: false Decision2: true Condition2.1: true Decision3: true Condition3.1: true
TC06	{1,1,2,2,3}	2	2 Pass	Path: Line1 → Line3 → Line4 → Line5 → Line6 → Line7 → Line8 → Line9 Decision1: false Decision2: true Condition2.1: true Decision3: true

CP353201 Software Quality Assurance (1/2568)

ผศ.ดร.ชิตสุธา สุ่มเล็ก

Lab instruction

				Condition3.1: true
TC07	{1,2,3,4}	0	0 Pass	Path: Line1 → Line3 → Line4 → Line5 → Line7 → Line8 → Line9 Decision1: false Decision2: false Condition2.1: false
TC08	{1,1,2,2,2, 3,3,4}	3	3 Pass	Path: Line1 → Line3 → Line4 → Line5 → Line6 → Line7 → Line8 → Line9 Decision1: false Decision2: true Condition2.1: true Decision3: true Condition3.1: true
TC09	{1,1,1,1}	1	1 Pass	Path: Line1 → Line3 → Line4 →

CP353201 Software Quality Assurance (1/2568)

ผศ.ดร.ชิตสุธา สุ่มเล็ก

Lab instruction

				Line5 → Line6 → Line9 Decision1: false Decision2: true Condition2.1: true
TC10	{1,2,1,2}	0	0 Pass	Path: Line1 → Line3 → Line4 → Line5 → Line7 → Line8 → Line9 Decision1: false Decision2: false Condition2.1: false
TC11	{1,1,2,2,3, 3,4,4,5,5}	5	5 Pass	Path: Line1 → Line3 → Line4 → Line5 → Line6 → Line7 → Line8 → Line9 Decision1: false Decision2: true Condition2.1: true Decision3: true

CP353201 Software Quality Assurance (1/2568)

ผศ.ดร.ชิตสุธา สุ่มเล็ก

Lab instruction

				Condition3.1: true
TC12	{1,2,3,4,5, 6,7,8,9}	0	0 Pass	Path: Line1 → Line3 → Line4 → Line5 → Line7 → Line8 → Line9 Decision1: false Decision2: false Condition2.1: false

C/DC coverage = (10/10) x 100% = 100%